

Tab 1

1 Group Employees by Department with Highest Salary in Each

```
import java.util.*;
import java.util.stream.*;

class Employee {
    String name;
    String dept;
    double salary;

    Employee(String name, String dept, double salary) {
        this.name = name;
        this.dept = dept;
        this.salary = salary;
    }

    public String getDept() { return dept; }
    public double getSalary() { return salary; }
    public String getName() { return name; }
}

public class Main {
    public static void main(String[] args) {

        List<Employee> employees = Arrays.asList(
            new Employee("John", "IT", 90000),
            new Employee("Mike", "IT", 120000),
            new Employee("Sara", "HR", 80000),
            new Employee("Anna", "HR", 95000)
        );

        Map<String, Optional<Employee>> result =
            employees.stream()
                .collect(Collectors.groupingBy(
                    Employee::getDept,
                    Collectors.maxBy(Comparator.comparing(Employee::getSalary))
                ));

        result.forEach((dept, emp) ->
            System.out.println(dept + " -> " + emp.get().getName()));
    }
}
```

```
}  
}
```

2 Find Duplicate Elements in a List

```
List<Integer> numbers = Arrays.asList(1,2,3,4,3,2,5);
```

```
Set<Integer> duplicates = numbers.stream()  
    .filter(n -> Collections.frequency(numbers, n) > 1)  
    .collect(Collectors.toSet());
```

```
System.out.println(duplicates);
```

👉 Optimized Version (Better Performance):

```
Set<Integer> seen = new HashSet<>();  
Set<Integer> duplicates = numbers.stream()  
    .filter(n -> !seen.add(n))  
    .collect(Collectors.toSet());
```

3 Partition Employees by Salary > 100000

```
Map<Boolean, List<Employee>> partition =  
    employees.stream()  
        .collect(Collectors.partitioningBy(e -> e.getSalary() > 100000));
```

```
System.out.println("High Earners: " + partition.get(true));
```

4 Top 3 Highest Salaries

```
employees.stream()  
    .sorted(Comparator.comparing(Employee::getSalary).reversed())  
    .limit(3)  
    .forEach(e -> System.out.println(e.getName()));
```

5 Convert List to Map (Handle Duplicate Keys)

```
Map<String, Double> empMap =
    employees.stream()
        .collect(Collectors.toMap(
            Employee::getDept,
            Employee::getSalary,
            (oldVal, newVal) -> newVal // merge function
        ));

System.out.println(empMap);
```

6 Flatten Nested List

```
List<List<String>> list = Arrays.asList(
    Arrays.asList("A", "B"),
    Arrays.asList("C", "D")
);

List<String> flat =
    list.stream()
        .flatMap(Collection::stream)
        .collect(Collectors.toList());

System.out.println(flat);
```

7 Find Second Highest Salary

```
Optional<Double> secondHighest =
    employees.stream()
        .map(Employee::getSalary)
        .distinct()
        .sorted(Comparator.reverseOrder())
        .skip(1)
        .findFirst();

System.out.println(secondHighest.orElse(0.0));
```

8 Count Occurrences of Each Word in String

```
String text = "java stream java api stream";
```

```
Map<String, Long> wordCount =  
    Arrays.stream(text.split(" "))  
        .collect(Collectors.groupingBy(  
            w -> w,  
            Collectors.counting()  
        ));
```

```
System.out.println(wordCount);
```

9 Find Longest String in List

```
Optional<String> longest =  
    list.stream()  
        .max(Comparator.comparing(String::length));
```

```
System.out.println(longest.orElse(""));
```

10 Parallel Stream with Performance Awareness

```
long count = employees.parallelStream()  
    .filter(e -> e.getSalary() > 90000)  
    .count();
```

```
System.out.println(count);
```

⚠ Senior Interview Tip:

- Use parallel streams only for CPU-intensive tasks.

- Avoid shared mutable state.
 - Ensure operations are stateless and associative.
-

11 Custom Collector (Advanced)

```
Collector<Employee, ?, Double> avgSalaryCollector =  
    Collectors.averagingDouble(Employee::getSalary);  
  
double avgSalary = employees.stream().collect(avgSalaryCollector);  
  
System.out.println(avgSalary);
```

12 Reduce Example (Total Salary)

```
double totalSalary =  
    employees.stream()  
        .map(Employee::getSalary)  
        .reduce(0.0, Double::sum);  
  
System.out.println(totalSalary);
```

Stream Questions

```
class Employee {  
  
    String name;  
  
    String email;  
  
    String dept;  
  
    String designation;  
  
    double salary;  
  
    // constructor + getters  
  
}
```

1 Department-wise Second Highest Salary

```
Map<String, Optional<Employee>> result =  
    employees.stream()  
        .collect(Collectors.groupingBy(
```

```

Employee::getDept,
Collectors.collectingAndThen(
    Collectors.toList(),
    list -> list.stream()
        .sorted(Comparator.comparing(Employee::getSalary).reversed())
        .skip(1)
        .findFirst()
)
));

```

2 Find Duplicate Employees by Email

```

Set<String> seen = new HashSet<>();
List<Employee> duplicates =
    employees.stream()
        .filter(e -> !seen.add(e.getEmail()))
        .toList();

```

3 Top 3 Employees Per Department

```

Map<String, List<Employee>> top3 =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDept,
            Collectors.collectingAndThen(
                Collectors.toList(),
                list -> list.stream()
                    .sorted(Comparator.comparing(Employee::getSalary).reversed())
                    .limit(3)
                    .toList()
            )
        ));

```

4 Salary > Department Average

```

Map<String, Double> avgSalary =
    employees.stream()
        .collect(Collectors.groupingBy(

```

```
Employee::getDept,  
Collectors.averagingDouble(Employee::getSalary)  
));
```

```
List<Employee> result =  
employees.stream()  
    .filter(e -> e.getSalary() > avgSalary.get(e.getDept()))  
    .toList();
```

5 Total Salary Per Department

```
Map<String, Double> total =  
employees.stream()  
    .collect(Collectors.groupingBy(  
        Employee::getDept,  
        Collectors.summingDouble(Employee::getSalary)  
    ));
```

6 Multi-Level Grouping

```
Map<String, Map<String, Long>> result =  
employees.stream()  
    .collect(Collectors.groupingBy(  
        Employee::getDept,  
        Collectors.groupingBy(  
            Employee::getDesignation,  
            Collectors.counting()  
        )  
    ));
```

7 Max Transaction Per Customer

```
Map<String, Optional<Transaction>> result =  
transactions.stream()  
    .collect(Collectors.groupingBy(  
        Transaction::getCustomer,  
        Collectors.maxBy(Comparator.comparing(Transaction::getAmount))  
    ));
```

8 Department with Highest Average Salary

```
Optional<Map.Entry<String, Double>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDept,
            Collectors.averagingDouble(Employee::getSalary)
        ))
        .entrySet()
        .stream()
        .max(Map.Entry.comparingByValue());
```

9 Partition by Salary Range

```
Map<String, List<Employee>> result =
    employees.stream()
        .collect(Collectors.groupingBy(e -> {
            if (e.getSalary() < 50000) return "LOW";
            else if (e.getSalary() <= 100000) return "MEDIUM";
            else return "HIGH";
        }));
```

10 Group Anagrams

```
Map<String, List<String>> result =
    words.stream()
        .collect(Collectors.groupingBy(word -> {
            char[] arr = word.toCharArray();
            Arrays.sort(arr);
            return new String(arr);
        }));
```

11 Any Department All Salary > 50k

```
boolean result =
```

```
employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept))
    .values()
    .stream()
    .anyMatch(list -> list.stream().allMatch(e -> e.getSalary() > 50000));
```

12 First Non-Repeated Character

```
Optional<Character> result =
    str.chars()
        .mapToObj(c -> (char)c)
        .collect(Collectors.groupingBy(
            c -> c,
            LinkedHashMap::new,
            Collectors.counting()
        ))
        .entrySet()
        .stream()
        .filter(e -> e.getValue() == 1)
        .map(Map.Entry::getKey)
        .findFirst();
```

13 Common Elements Between Two Lists

```
List<Integer> result =
    list1.stream()
        .filter(list2::contains)
        .distinct()
        .toList();
```

14 Remove Duplicates Preserve Order

```
List<Integer> result =
    list.stream()
        .distinct()
        .toList();
```

15 Longest Consecutive Sequence

```
int longest =  
    list.stream()  
        .sorted()  
        .reduce(new int[]{0, 0, Integer.MIN_VALUE}, (a, b) -> {  
            if (b == a[2] + 1) a[0]++;  
            else a[0] = 1;  
            a[1] = Math.max(a[1], a[0]);  
            a[2] = b;  
            return a;  
        }, (a, b) -> a)[1];
```

16 Top 10 Highest Without Full Sort

```
PriorityQueue<Employee> pq =  
    employees.stream()  
        .collect(Collectors.toCollection(  
            () -> new PriorityQueue<>(Comparator.comparing(Employee::getSalary))  
        ));  
  
while (pq.size() > 10) pq.poll();  
  
List<Employee> top10 = new ArrayList<>(pq);
```

17 Parallel Stream Safe Collect

✗ Wrong:

```
List<Employee> result = new ArrayList<>();  
employees.parallelStream().forEach(result::add);
```

✓ Correct:

```
List<Employee> result =  
    employees.parallelStream().collect(Collectors.toList());
```

18 First 100 Even Numbers After Squaring

```
List<Integer> result =  
    numbers.stream()  
        .map(n -> n * n)  
        .filter(n -> n % 2 == 0)  
        .limit(100)  
        .toList();
```

19 Custom Collector Join with Prefix/Suffix

```
String result =  
    names.stream()  
        .collect(Collectors.joining(", ", "Employees: ", "."));
```

20 Product of All Integers

```
int product =  
    list.stream()  
        .reduce(1, (a, b) -> a * b);
```

21 Manual Average Using Reduce

```
double avg =  
    list.stream()  
        .reduce(new double[]{0,0},  
            (a,b) -> { a[0]+=b; a[1]++; return a; },  
            (a,b) -> a)  
        .get()[0] / list.size();
```

22 Total Revenue Per Product (Nested)

```
Map<String, Double> revenue =
```

```
orders.stream()
    .flatMap(order -> order.getItems().stream())
    .collect(Collectors.groupingBy(
        Item::getProduct,
        Collectors.summingDouble(Item::getPrice)
    ));
```

23 Flatten and Group by Category

```
Map<String, Integer> result =
    customers.stream()
        .flatMap(c -> c.getOrders().stream())
        .flatMap(o -> o.getItems().stream())
        .collect(Collectors.groupingBy(
            Item::getCategory,
            Collectors.summingInt(Item::getQuantity)
        ));
```

24 Customer Who Spent Maximum

```
Optional<Map.Entry<String, Double>> result =
    orders.stream()
        .collect(Collectors.groupingBy(
            Order::getCustomer,
            Collectors.summingDouble(Order::getAmount)
        ))
        .entrySet()
        .stream()
        .max(Map.Entry.comparingByValue());
```

25 Log File Analysis

```
Map<String, Long> errorCount =
    logs.stream()
        .filter(log -> log.getLevel().equals("ERROR"))
        .collect(Collectors.groupingBy(
            Log::getService,
            Collectors.counting()
        ));
```

```
));
```

```
List<Log> top5RecentErrors =
```

```
logs.stream()
```

```
    .filter(log -> log.getLevel().equals("ERROR"))
```

```
    .sorted(Comparator.comparing(Log::getTimestamp).reversed())
```

```
    .limit(5)
```

```
    .toList();
```

CompletableFuture

1 Parallel REST Calls Using CompletableFuture + Stream

Problem:

Call multiple microservices in parallel and collect results.

```
import java.util.*;
import java.util.concurrent.*;
import java.util.stream.*;

public class Main {

    static ExecutorService executor = Executors.newFixedThreadPool(5);

    static String callService(String service) {
        try { Thread.sleep(1000); } catch (Exception e){}
        return service + " response";
    }

    public static void main(String[] args) {

        List<String> services = List.of("User", "Order", "Payment");

        List<CompletableFuture<String>> futures =
            services.stream()
                .map(service ->
                    CompletableFuture.supplyAsync(
                        () -> callService(service), executor))
                .toList();

        List<String> responses =
            futures.stream()
                .map(CompletableFuture::join)
                .toList();

        System.out.println(responses);
        executor.shutdown();
    }
}
```

💡 Interview Discussion:

- Why not parallelStream?
 - Why custom executor?
 - What happens if one service fails?
-

🔥 2 Combine Multiple Async Calls

```
CompletableFuture<String> userFuture =  
    CompletableFuture.supplyAsync(() -> "User Data");  
  
CompletableFuture<String> orderFuture =  
    CompletableFuture.supplyAsync(() -> "Order Data");  
  
CompletableFuture<String> combined =  
    userFuture.thenCombine(orderFuture,  
        (user, order) -> user + " + " + order);  
  
System.out.println(combined.join());
```

🔥 3 Fetch Data, Transform Using Stream, Return Async

```
CompletableFuture<List<String>> result =  
    CompletableFuture.supplyAsync(() -> List.of("java", "stream", "api"))  
        .thenApply(list ->  
            list.stream()  
                .map(String::toUpperCase)  
                .toList()  
        );  
  
System.out.println(result.join());
```

4 Handle Exceptions Properly

```
CompletableFuture<String> future =  
    CompletableFuture.supplyAsync(() -> {  
        if(true) throw new RuntimeException("Service Failed");  
        return "Success";  
    }).exceptionally(ex -> "Fallback Response");  
  
System.out.println(future.join());
```

💡 Senior Tip:
Use:

- `handle()`
 - `exceptionally()`
 - `whenComplete()`
-

5 Run Multiple Async Tasks and Wait for All

```
List<CompletableFuture<String>> futures =  
    List.of("A", "B", "C").stream()  
        .map(s -> CompletableFuture.supplyAsync(() -> s + " Done"))  
        .toList();
```

```
CompletableFuture<Void> all =  
    CompletableFuture.allOf(  
        futures.toArray(new CompletableFuture[0])  
    );
```

```
all.join();
```

```
List<String> result =  
    futures.stream()  
        .map(CompletableFuture::join)  
        .toList();
```

```
System.out.println(result);
```

6 First Completed Task (Race Condition Handling)

```
CompletableFuture<String> f1 =  
    CompletableFuture.supplyAsync(() -> {  
        sleep(2000); return "Slow";  
    });  
  
CompletableFuture<String> f2 =  
    CompletableFuture.supplyAsync(() -> {  
        sleep(1000); return "Fast";  
    });  
  
String winner =  
    f1.applyToEither(f2, result -> result).join();  
  
System.out.println(winner);
```

7 Advanced: Async DB Calls + Aggregation

Scenario:

- Fetch Orders
- Fetch Payments
- Merge
- Calculate total revenue

```
CompletableFuture<List<Integer>> ordersFuture =  
    CompletableFuture.supplyAsync(() -> List.of(100, 200, 300));
```

```
CompletableFuture<List<Integer>> paymentsFuture =
    CompletableFuture.supplyAsync(() -> List.of(50, 150));

CompletableFuture<Integer> total =
    ordersFuture.thenCombine(paymentsFuture, (orders, payments) -> {
        int sumOrders = orders.stream().mapToInt(Integer::intValue).sum();
        int sumPayments = payments.stream().mapToInt(Integer::intValue).sum();
        return sumOrders + sumPayments;
    });

System.out.println(total.join());
```

8 Timeout Handling (Very Important in Interviews)

```
CompletableFuture<String> future =
    CompletableFuture.supplyAsync(() -> {
        sleep(3000);
        return "Response";
    })
    .orTimeout(2, TimeUnit.SECONDS)
    .exceptionally(ex -> "Timeout Fallback");

System.out.println(future.join());
```

9 Custom Executor + Performance Control

```
ExecutorService executor =
    new ThreadPoolExecutor(
        2, 4,
        60, TimeUnit.SECONDS,
        new LinkedBlockingQueue<>(10)
    );

CompletableFuture<String> future =
    CompletableFuture.supplyAsync(() -> "Task", executor);
```

```
System.out.println(future.join());
executor.shutdown();
```

💡 Interview Point:

- Avoid commonForkJoinPool in heavy IO apps
 - Always control thread pool
-

10 Real Enterprise Pattern: Async Pipeline

```
CompletableFuture.supplyAsync(() -> fetchUser())
    .thenApply(user -> enrichUser(user))
    .thenCompose(user -> fetchOrdersAsync(user))
    .thenApply(orders ->
        orders.stream()
            .filter(o -> o > 100)
            .toList()
    )
    .thenAccept(System.out::println)
    .join();
```

Optional Inside Stream

1 Optional Inside Stream → Avoid Optional<Optional<T>>

? Problem

Find first employee with salary > 100k.

```
Optional<Optional<Employee>> result =  
    Optional.of(  
        employees.stream()  
            .filter(e -> e.getSalary() > 100000)  
            .findFirst()  
    );
```

⚠ Problem: You get `Optional<Optional<Employee>>`

✓ Correct

```
Optional<Employee> result =  
    employees.stream()  
        .filter(e -> e.getSalary() > 100000)  
        .findFirst();
```

🧠 Interview test:

Do you understand when NOT to wrap Optional?

🔥 2 Optional Field Inside Object (Null-Safe Filtering)

```
class Employee {  
    Optional<String> email;  
}
```

? Find employees with gmail accounts

⚠ Common mistake:

```
employees.stream()
    .filter(e -> e.getEmail().get().contains("gmail"))
```

💥 Throws NoSuchElementException

✓ Correct

```
employees.stream()
    .map(Employee::getEmail)
    .flatMap(Optional::stream)
    .filter(email -> email.contains("gmail"))
    .toList();
```

🧠 Interview test:

Do you know `Optional::stream` (Java 9+)?

🔥 3 Null List + Stream = NPE

? Safe stream on possibly null list

⚠ Wrong:

```
list.stream()
```

✓ Safe Version

```
Optional.ofNullable(list)
    .orElseGet(Collections::emptyList)
    .stream()
    .forEach(System.out::println);
```

🧠 Testing defensive programming.

4 Optional Chaining with map vs flatMap

```
Optional<Employee> emp = findEmployee();
```

? Get employee's manager's email

⚠ Wrong:

```
emp.map(Employee::getManager)
    .map(Manager::getEmail)
```

If `getManager()` returns `Optional` → nesting issue

✓ Correct

```
emp.flatMap(Employee::getManager)
    .flatMap(Manager::getEmail)
    .orElse("Not Found");
```

🧠 Interview test:

`map()` vs `flatMap()` difference

5 `orElse` vs `orElseGet` (Performance Trap)

```
Optional<String> opt = Optional.of("Hello");
```

⚠ Tricky:

```
opt.orElse(expensiveCall());
```

👉 `expensiveCall()` runs even if value present

✓ Correct

```
opt.orElseGet(() -> expensiveCall());
```

🧠 Interview test:
Lazy evaluation understanding

🔥 6 Filtering Empty Optionals in Stream

```
List<Optional<String>> list =  
    List.of(Optional.of("A"), Optional.empty(), Optional.of("B"));
```

? Extract values safely

✓ Best Way (Java 9+)

```
List<String> result =  
    list.stream()  
        .flatMap(Optional::stream)  
        .toList();
```

🧠 Interview test:
Elegant Optional flattening

🔥 7 Reduce Returning Optional

```
Optional<Integer> sum =  
    list.stream().reduce(Integer::sum);
```

⚠ Why Optional?
Because stream may be empty.

✓ Safe Handling

```
int result =
```

```
list.stream()
    .reduce(Integer::sum)
    .orElse(0);
```

🧠 Interview test:
Empty stream awareness

🔥 8 Optional in Map Value (Advanced)

Map<String, Optional<Employee>> map;

? Get all present employees



```
List<Employee> result =
    map.values().stream()
        .flatMap(Optional::stream)
        .toList();
```

🔥 9 Returning Optional from Stream Operation

⚠ Anti-pattern:

```
Optional<Employee> result =
    Optional.of(
        employees.stream()
            .filter(...)
            .findFirst()
    ).flatMap(e -> e);
```

Overcomplicated.

✓ Just:

```
employees.stream().filter(...).findFirst();
```

🧠 Testing simplicity mindset.

🔥 10 Nested Optionals + Stream (Hard)

```
class Order {  
    Optional<Customer> customer;  
}
```

? Get all customer emails from orders



```
orders.stream()  
    .map(Order::getCustomer)  
    .flatMap(Optional::stream)  
    .map(Customer::getEmail)  
    .flatMap(Optional::stream)  
    .toList();
```

🧠 Testing deep flattening

🔥 11 Optional + Collectors.toMap Trap

⚠ This throws if value is empty:

```
.collect(Collectors.toMap(  
    Employee::getId,  
    e -> e.getEmail().get()  
));
```

✓ Safe

```
.collect(Collectors.toMap(  
    Employee::getId,  
    e -> e.getEmail().orElse("N/A")  
));
```

12 Optional.filter() Trick

```
Optional<String> name = Optional.of("John");  
name.filter(n -> n.startsWith("J"))  
    .ifPresent(System.out::println);
```

💡 If filter fails → Optional becomes empty

13 Optional vs Stream.findAny()

```
Optional<Employee> emp =  
    employees.stream().findAny();
```

⚠ In parallel stream → non-deterministic

14 Optional in Comparator (Hard)

⚠ Comparing optional values:

```
Comparator<Employee> comp =  
    Comparator.comparing(  
        e -> e.getEmail().orElse("")  
    );
```

Better:

```
Comparator<Employee> comp =  
    Comparator.comparing(  
        e -> e.getEmail().orElse(""),  
        Comparator.nullsLast(String::compareTo)  
    );
```

Optional + Exception Throwing

```
Employee emp =  
    employees.stream()  
        .filter(...)  
        .findFirst()  
        .orElseThrow(() -> new RuntimeException("Not Found"));
```

 Testing proper error handling

Java Collections In-Depth

Java Collections In-Depth

1 Collections Framework Overview

- Hierarchy:

Collection (List, Set, Queue)
Map

- Interfaces:
 - **Collection** → root interface for List, Set, Queue
 - **List** → ordered, duplicates allowed
 - **Set** → unordered, no duplicates
 - **Queue / Deque** → FIFO/LIFO
 - **Map** → key-value mapping
- Implementations:
 - `ArrayList`, `LinkedList`, `HashSet`, `TreeSet`, `HashMap`, `TreeMap`, `LinkedHashMap`, `PriorityQueue`, `ConcurrentHashMap`, etc.
- Utilities:
 - `Collections` → static methods: sort, reverse, shuffle
 - `Arrays` → utility for arrays

2 List Interface

Feature	ArrayList	LinkedList
Storage	Dynamic array	Doubly linked list
Random access	O(1)	O(n)
Insertion/removal	O(n)	O(1) (at ends)
Memory overhead	Low	High
Thread safe	No	No

Example: ArrayList

```
List<String> list = new ArrayList<>();
list.add("Java");
list.add("Stream");
list.add("Collections");

for(String s : list) System.out.println(s);
```

3 Set Interface

- **HashSet** → backed by **HashMap**, unordered, no duplicates
- **LinkedHashSet** → maintains insertion order
- **TreeSet** → ordered (sorted), uses **TreeMap**, O(log n) operations

Example: HashSet

```
Set<String> set = new HashSet<>();
set.add("A");
set.add("B");
set.add("A"); // ignored
```

Performance:

Operation	HashSe t	TreeSet
add	O(1)	O(log n)

remove	O(1)	O(log n)
contains	O(1)	O(log n)

4 Map Interface

- **HashMap** → unordered, allows null key/value, not thread-safe
- **LinkedHashMap** → maintains insertion order
- **TreeMap** → sorted order (keys must implement Comparable)
- **ConcurrentHashMap** → thread-safe

Example: HashMap

```
Map<String, Integer> map = new HashMap<>();
map.put("Java", 10);
map.put("Python", 20);

map.forEach((k,v) -> System.out.println(k + "=" + v));
```

Performance:

Operation	HashMap	TreeMap
get	O(1)	O(log n)
put	O(1)	O(log n)

5 Queue and Deque

- **Queue** → FIFO (LinkedList, PriorityQueue)
- **Deque** → double-ended (ArrayDeque, LinkedList)

Example: PriorityQueue

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

```
pq.add(5);  
pq.add(1);  
pq.add(10);
```

```
while(!pq.isEmpty()) System.out.println(pq.poll());
```

- Uses **heap** internally
 - $O(\log n)$ for insertion and removal
-

6 Collections Utility Class

- Sorting, reversing, min/max, shuffle
- Thread-safe wrappers: `Collections.synchronizedList()`

```
List<String> list = new ArrayList<>();  
Collections.addAll(list, "C", "A", "B");  
Collections.sort(list); // A B C
```

7 Arrays Utility Class

- `Arrays.asList()`
- `Arrays.sort()`
- `Arrays.binarySearch()`
- Deep operations: `Arrays.deepEquals()`, `Arrays.deepToString()`

```
String[] arr = {"Z", "A", "M"};  
Arrays.sort(arr); // [A,M,Z]
```

8 Concurrent Collections

Collection	Feature
ConcurrentHashMap	Thread-safe hash map
CopyOnWriteArrayList	Safe iteration during modification
BlockingQueue	Thread-safe queue with blocking support

Example: ConcurrentHashMap

```
ConcurrentHashMap<String, Integer> cmap = new ConcurrentHashMap<>();  
cmap.put("Java", 1);  
cmap.putIfAbsent("Python", 2);
```

9 Advanced Topics

✓ 1. Fail-Fast vs Fail-Safe

- **Fail-Fast:** ArrayList iterator → throws `ConcurrentModificationException` if modified during iteration
 - **Fail-Safe:** CopyOnWriteArrayList iterator → works on snapshot
-

✓ 2. Performance Pitfalls

- Using LinkedList when frequent random access needed → $O(n)$
 - Using synchronized wrapper unnecessarily → contention
 - Using streams over primitive wrapper types → boxing overhead
-

✓ 3. Memory Considerations

- HashMap: default capacity = 16, load factor = 0.75

- TreeMap: node objects overhead
 - Linked structures → memory heavy due to next/prev references
-

✓ 4. Key Interview Traps

- Null key/value differences (HashMap vs TreeMap)
 - Iterating with forEach vs Iterator performance
 - Map merging with `merge()` and `computeIfAbsent()`
 - equals() & hashCode() implementation correctness
-

10 Best Practices for Senior Developers

- Prefer `ArrayList` over `LinkedList` unless insertion/removal frequent
 - Use primitive streams for numeric operations to avoid boxing
 - Use concurrent collections for multi-threaded access
 - Avoid excessive synchronized wrappers
 - Prefer `Collectors.toMap()` or `groupingByConcurrent()` for stream aggregations
 - Understand trade-offs: memory vs speed vs thread-safety
-

11 Quick Reference Table

Interface	Implementation	Complexity	Ordered	Thread-Safe
List	ArrayList	get O(1), add O(1 amortized)	Yes	No

List	LinkedList	get O(n), add O(1)	Yes	No
Set	HashSet	add/remove O(1)	No	No
Set	TreeSet	add/remove O(log n)	Sorted	No
Map	HashMap	get/put O(1)	No	No
Map	TreeMap	get/put O(log n)	Sorted	No
Map	ConcurrentHashMap	get/put O(1)	No	Yes

✓ This PDF-ready guide is structured for **printing, quick interview prep, and deep reference**.

I can generate it **directly as a PDF with formatting and code highlighting**.

Do you want me to create the **actual downloadable PDF file** next?

Collections in depth with coding ready to pdf format

Perfect! I can prepare a **full “Java Collections In-Depth” guide with code examples**, properly formatted, so it can be exported as a PDF.

Here’s the **ready-to-export version**, structured for clarity, with code blocks, tables, and senior-level notes.

Java Collections In-Depth – Senior Developer Guide

1 Collections Framework Overview

Hierarchy

Collection

- ├── List
- ├── Set
- └── Queue

Map

Interfaces:

- **Collection** → Root interface (List, Set, Queue)
- **List** → Ordered, duplicates allowed
- **Set** → Unordered, no duplicates
- **Queue/Deque** → FIFO / LIFO
- **Map** → Key-Value mapping

Utilities:

- **Collections** → static helper methods
- **Arrays** → array utilities

2 List Interface

Feature	ArrayList	LinkedList
Storage	Dynamic array	Doubly linked list
Random access	O(1)	O(n)
Insertion/removal	O(n)	O(1) (at ends)
Memory overhead	Low	High
Thread safe	No	No

Example: ArrayList

```
List<String> list = new ArrayList<>();  
list.add("Java");  
list.add("Stream");  
list.add("Collections");
```

```
for(String s : list) {  
    System.out.println(s);  
}
```

3 Set Interface

- **HashSet:** Unordered, no duplicates
- **LinkedHashSet:** Maintains insertion order
- **TreeSet:** Sorted (uses **TreeMap**), $O(\log n)$

Example: HashSet

```
Set<String> set = new HashSet<>();  
set.add("A");  
set.add("B");  
set.add("A"); // ignored
```

```
set.forEach(System.out::println);
```

Performance:

Operation	HashSe t	TreeSet
add	$O(1)$	$O(\log n)$
remove	$O(1)$	$O(\log n)$
contains	$O(1)$	$O(\log n)$

4 Map Interface

- **HashMap:** Unordered, allows null key/value
- **LinkedHashMap:** Maintains insertion order
- **TreeMap:** Sorted keys (Comparable)

- **ConcurrentHashMap**: Thread-safe

Example: HashMap

```
Map<String, Integer> map = new HashMap<>();
map.put("Java", 10);
map.put("Python", 20);

map.forEach((k,v) -> System.out.println(k + "=" + v));
```

Performance:

Operation	HashMap	TreeMap
get	$O(1)$	$O(\log n)$
put	$O(1)$	$O(\log n)$

5 Queue and Deque

- **Queue** → FIFO (LinkedList, PriorityQueue)
- **Deque** → double-ended (ArrayDeque, LinkedList)

Example: PriorityQueue

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(5);
pq.add(1);
pq.add(10);

while(!pq.isEmpty()) {
    System.out.println(pq.poll());
}
```

- Internally uses a **heap**
 - $O(\log n)$ insertion/removal
-

6 Collections Utility Class

- Sorting, reversing, min/max, shuffle
- Thread-safe wrappers: `Collections.synchronizedList()`

```
List<String> list = new ArrayList<>();  
Collections.addAll(list, "C", "A", "B");  
Collections.sort(list); // A B C
```

7 Arrays Utility Class

- `Arrays.asList()`
- `Arrays.sort()`
- `Arrays.binarySearch()`
- Deep operations: `Arrays.deepEquals()`, `Arrays.deepToString()`

```
String[] arr = {"Z","A","M"};  
Arrays.sort(arr); // [A,M,Z]  
System.out.println(Arrays.toString(arr));
```

8 Concurrent Collections

Collection	Feature
ConcurrentHashMap	Thread-safe hash map
CopyOnWriteArrayList	Safe iteration during modification
BlockingQueue	Thread-safe queue with blocking support

Example: ConcurrentHashMap

```
ConcurrentHashMap<String, Integer> cmap = new ConcurrentHashMap<>();
```

```
cmap.put("Java", 1);  
cmap.putIfAbsent("Python", 2);
```

9 Advanced Topics

Fail-Fast vs Fail-Safe

- Fail-Fast → ArrayList iterator → throws `ConcurrentModificationException`
- Fail-Safe → CopyOnWriteArrayList → works on snapshot

Performance Pitfalls

- LinkedList for random access → $O(n)$
- Synchronized wrapper overuse → contention
- Streams over boxed types → autoboxing overhead

Memory Considerations

- HashMap default capacity = 16, load factor = 0.75
- Linked structures → high memory due to next/prev pointers

Interview Traps

- Null key/value differences (HashMap vs TreeMap)
 - Iteration performance (forEach vs Iterator)
 - Map merging: `merge()` vs `computeIfAbsent()`
 - Proper `equals()` & `hashCode()` implementations
-

10 Best Practices

- Prefer `ArrayList` unless insertion/removal frequent
 - Use primitive streams to avoid boxing
 - Use concurrent collections for multithreading
 - Avoid unnecessary synchronized wrappers
 - Stream aggregations → prefer `Collectors.toMap()` or `groupingByConcurrent()`
 - Understand trade-offs: memory vs speed vs thread-safety
-

11 Quick Reference Table

Interface	Implementation	Complexity	Ordered	Thread-Safe
List	ArrayList	get $O(1)$, add $O(1)$ amortized)	Yes	No
List	LinkedList	get $O(n)$, add $O(1)$	Yes	No
Set	HashSet	add/remove $O(1)$	No	No
Set	TreeSet	add/remove $O(\log n)$	Sorted	No
Map	HashMap	get/put $O(1)$	No	No
Map	TreeMap	get/put $O(\log n)$	Sorted	No
Map	ConcurrentHashMap	get/put $O(1)$	No	Yes

Collections coding

1 Duplicate Removal While Preserving Order

? Problem

Remove duplicates from a `List<String>` while maintaining the original insertion order.

```
List<String> list = List.of("Java", "Python", "Java", "C++", "Python");
```

✓ Solution

```
List<String> result = new ArrayList<>(  
    new LinkedHashSet<>(list)  
);  
System.out.println(result); // [Java, Python, C++, Python]
```

🧠 Test: Understanding of `Set` types and insertion order.

2 Find Top N Frequent Elements

? Problem

Given a `List<Integer>` `nums` and an integer `N`, find the `N` most frequent numbers.

```
List<Integer> nums = List.of(1,1,1,2,2,3,3,3,4);  
int N = 2;
```

✓ Solution

```
Map<Integer, Long> freq = nums.stream()  
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));  
  
List<Integer> topN = freq.entrySet().stream()  
    .sorted(Map.Entry.<Integer, Long>comparingByValue().reversed())  
    .limit(N)  
    .map(Map.Entry::getKey)  
    .toList();
```

```
System.out.println(topN); // [3,1]
```

🧠 Test: Map aggregation, sorting, functional streams.

3 Merge Two Maps With Summed Values

? Problem

Given two maps:

```
Map<String, Integer> map1 = Map.of("A", 1, "B", 2);  
Map<String, Integer> map2 = Map.of("B", 3, "C", 4);
```

Merge them summing the values of common keys.

✓ Solution

```
Map<String, Integer> merged = new HashMap<>(map1);  
map2.forEach((k,v) -> merged.merge(k, v, Integer::sum));  
System.out.println(merged); // {A=1, B=5, C=4}
```

🧠 Test: `Map.merge()` knowledge, functional coding.

4 Reverse a LinkedHashMap

? Problem

Given a `LinkedHashMap<String, Integer>` maintain insertion order, reverse the order.

```
LinkedHashMap<String, Integer> map = new LinkedHashMap<>();  
map.put("A",1); map.put("B",2); map.put("C",3);
```

✓ Solution

```
LinkedHashMap<String, Integer> reversed = new LinkedHashMap<>();  
new ArrayList<>(map.entrySet()).stream()  
    .sorted((e1,e2) -> -1) // reverse order  
    .forEach(e -> reversed.put(e.getKey(), e.getValue()));
```

```
System.out.println(reversed); // {C=3, B=2, A=1}
```

🧠 Test: LinkedHashMap ordering, stream operations.

5 Find Missing Numbers in a Range

? Problem

Given a `List<Integer>` containing numbers from 1 to N with some missing, find the missing numbers.

```
List<Integer> nums = List.of(1,2,4,6);  
int N = 6;
```

✓ Solution

```
Set<Integer> set = new HashSet<>(nums);  
List<Integer> missing = IntStream.rangeClosed(1, N)  
    .filter(i -> !set.contains(i))  
    .boxed()  
    .toList();  
System.out.println(missing); // [3,5]
```

🧠 Test: Set for fast lookup, IntStream.

6 Count Occurrences Using Map

```
List<String> words = List.of("apple", "banana", "apple", "orange", "banana", "apple");
```

✓ Solution

```
Map<String, Long> count = words.stream()  
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));  
System.out.println(count); // {orange=1, banana=2, apple=3}
```

🧠 Test: `Collectors.groupingBy` and counting.

7 Sort Map by Value

```
Map<String, Integer> map = Map.of("A",5, "B",1, "C",3);
```

✓ Solution

```
LinkedHashMap<String,Integer> sorted = map.entrySet().stream()
    .sorted(Map.Entry.comparingByValue())
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        (a,b) -> b,
        LinkedHashMap::new
    ));
System.out.println(sorted); // {B=1, C=3, A=5}
```

🧠 Test: Stream sorting and map collection.

8 Group Employees by Department

```
class Employee {
    String name;
    String dept;
    Employee(String name, String dept){this.name=name;this.dept=dept;}
}
List<Employee> emps = List.of(
    new Employee("John","HR"),
    new Employee("Jane","IT"),
    new Employee("Jim","HR")
);
```

✓ Solution

```
Map<String, List<Employee>> grouped = emps.stream()
    .collect(Collectors.groupingBy(e -> e.dept));
System.out.println(grouped.keySet()); // [HR, IT]
```

🧠 Test: `Collectors.groupingBy`.

9 Detect Duplicate Elements in List

```
List<Integer> nums = List.of(1,2,3,2,4,5,1);
```

✓ Solution

```
Set<Integer> seen = new HashSet<>();
Set<Integer> duplicates = nums.stream()
    .filter(n -> !seen.add(n))
    .collect(Collectors.toSet());
System.out.println(duplicates); // [1,2]
```

🧠 Test: HashSet for O(1) lookup, stream filtering.

10 Top N Elements by Custom Comparator

```
List<Employee> emps = List.of(
    new Employee("John", "HR", 4000),
    new Employee("Jane", "IT", 6000),
    new Employee("Jim", "HR", 5000)
);
```

✓ Solution

```
List<Employee> top2 = emps.stream()
    .sorted(Comparator.comparingInt(Employee::getSalary).reversed())
    .limit(2)
    .toList();
```

🧠 Test: Comparator, sorting, limit, streams.

11 Convert List to Map Safely

```
List<Employee> emps = List.of(
    new Employee("John", "HR"),
    new Employee("Jane", "IT"),
    new Employee("Jim", "HR")
);
```

```
);
```

✓ Solution

```
Map<String, Employee> map = emps.stream()
    .collect(Collectors.toMap(
        e -> e.name,
        Function.identity(),
        (existing, replacement) -> existing
    ));
```

🧠 Test: Handling duplicate keys in `Collectors.toMap()`.

12 Find Intersection of Two Lists

```
List<String> list1 = List.of("A","B","C");
List<String> list2 = List.of("B","C","D");
```

✓ Solution

```
List<String> intersection = list1.stream()
    .filter(list2::contains)
    .toList();
System.out.println(intersection); // [B,C]
```

🧠 Test: Stream filtering, set operations.

13 Sliding Window Sum Using Deque

```
int[] nums = {1,2,3,4,5};
int k = 3;
```

✓ Solution

```
Deque<Integer> window = new ArrayDeque<>();
List<Integer> sums = new ArrayList<>();
int sum=0;
for(int i=0;i<nums.length;i++){
    window.add(nums[i]);
    sum += nums[i];
```

```
if(window.size() > k) sum -= window.removeFirst();
if(window.size() == k) sums.add(sum);
}
System.out.println(sums); // [6,9,12]
```

🧠 Test: Knowledge of Deque, sliding window patterns.

14 Merge Two Sorted Lists

```
List<Integer> l1 = List.of(1,3,5);
List<Integer> l2 = List.of(2,4,6);
```

✓ Solution

```
List<Integer> merged = Stream.concat(l1.stream(), l2.stream())
    .sorted()
    .toList();
System.out.println(merged); // [1,2,3,4,5,6]
```

🧠 Test: Streams, merging, sorting.

15 Implement LRU Cache with LinkedHashMap

```
int capacity = 3;
LinkedHashMap<Integer,String> cache = new LinkedHashMap<>(capacity, 0.75f, true){
    protected boolean removeEldestEntry(Map.Entry<Integer,String> eldest){
        return size() > capacity;
    }
};
cache.put(1,"A"); cache.put(2,"B"); cache.put(3,"C");
cache.get(1); // access 1
cache.put(4,"D"); // remove LRU
System.out.println(cache.keySet()); // [3,1,4]
```

🧠 Test: Advanced LinkedHashMap usage, LRU logic.

This set of **15+ senior-level coding challenges** covers:

- Maps, Sets, Lists, Queues, Deques
- Stream + Collector operations
- Sorting, grouping, deduplication
- Concurrent collections, performance-aware coding
- Classic LRU cache problem